



IIC2343 - Arquitectura de Computadores (I/2026)

Ayudantía 9: RISC-V

Ayudantes: Alberto Maturana (alberto.maturana@uc.cl), Mario Rojas
(mario.denzel@estudiante.uc.cl), Fernanda Escobar (fer_jez@uc.cl)

Pregunta 1: Encuesta de Casas de Hogwarts

Usted desea determinar cuál de las cuatro casas de Hogwarts es la mejor basándose en una encuesta a 12 amigos. Cada amigo votó por una opción (1: Gryffindor, 2: Ravenclaw, 3: Hufflepuff, 4: Slytherin).

Implemente un programa en RISC-V que recorra la lista de votos almacenada en memoria, cuente las frecuencias de cada casa y finalmente imprima el número de la casa ganadora.

Resultados: [4, 3, 1, 4, 2, 2, 1, 3, 4, 1, 2, 4]

Paso Intermedio (Python): Antes de programar en ensamblador, revise la lógica en Python para ayudarte a pensar

```
# Lógica intermedia para la encuesta
votos = [4, 3, 1, 4, 2, 2, 1, 3, 4, 1, 2, 4]
# Contadores para casas 1, 2, 3 y 4
conteos = [0, 0, 0, 0]

# Contar frecuencias
for v in votos:
    conteos[v-1] += 1

# Encontrar el ganador (ID de la casa)
max_votos = 0
ganador = 0
for i in range(4):
    if conteos[i] > max_votos:
        max_votos = conteos[i]
        ganador = i + 1

print(ganador)
```

Hint: Utilice la directiva `.word` para almacenar los votos en el segmento `.data`. Dado que cada instrucción y palabra ocupa 4 bytes, recuerde ajustar sus desplazamientos de memoria en consecuencia.

Pregunta 2: Explique el código

En el siguiente fragmento de código se realiza el llamado de una subrutina `func_n_m`:

```
.data
n:  .word 37
m:  .word 5
q:  .word 0

.text
main:
    addi sp, sp, -16
    sw   ra, 12(sp)

    la   t0, n
    lw   a0, 0(t0)
    la   t1, m
    lw   a1, 0(t1)

    jal  ra, func_n_m

    la   t2, q
    sw   a0, 0(t2)

    lw   ra, 12(sp)
    addi sp, sp, 16
    addi a7, zero, 10
    ecall

func_n_m:
    blt  a0, a1, ccc

    addi sp, sp, -16
    sw   ra, 12(sp)

    sub  a0, a0, a1
    jal  ra, func_n_m
    addi a0, a0, 1

    lw   ra, 12(sp)
    addi sp, sp, 16
    jalr x0, 0(ra)

ccc:
    addi a0, x0, 0
    jalr x0, 0(ra)
```

Este fragmento representa el cómputo de una función $f(n, m)$. A partir de este:

1. (1.5 ptos.) Indique, con argumentos y en términos de n y m , lo que retorna la función $f(n, m)$.
Por ejemplo, $f(n, m) = n + m$.
2. (1.5 ptos.) Indique, con argumentos, si el fragmento anterior respeta o no la convención de llamadas de RISC-V.

Pregunta 3: Elabore el código

Elabore, utilizando el Assembly RISC-V, un programa que, dado un arreglo `arr` de largo `len` cuyos elementos son enteros de 32 bits (`.word`), determine si este es un “palíndromo” (1) o no (0), es decir, que si se invierte el orden de sus elementos, el arreglo no cambia. El resultado se debe computar en la variable `is_palindrome`.

A continuación, tres ejemplos:

```
arr = [1, 3, 3, 1] → is_palindrome = 1
arr = [1, 2, 3, 2, 1] → is_palindrome = 1
arr = [1, 2, 3, 4, 5, 6] → is_palindrome = 0
```

Puede utilizar el siguiente fragmento de código como base:

```
.data
    arr:          .word 1,3,3,1 # Arreglo
    len:          .word 4      # Largo del arreglo
    is_palindrome: .word -1     # Es palíndromo => TRUE = 1, FALSE = 0
.text
    # Su código aquí
```

IMPORTANTE: No es necesario que respete la convención RISC-V en este ejercicio.

Feedback ayudantía

Escanee el QR para entregar feedback sobre la ayudantía.

